Zürcher Hochschule
für Angewandte Wissenschaften
zh aw School of Engineering
InES Institute of Embedded Systems

Embedded Software Engineering
Software Paradigms

Simon Künzli, Tobias Welti

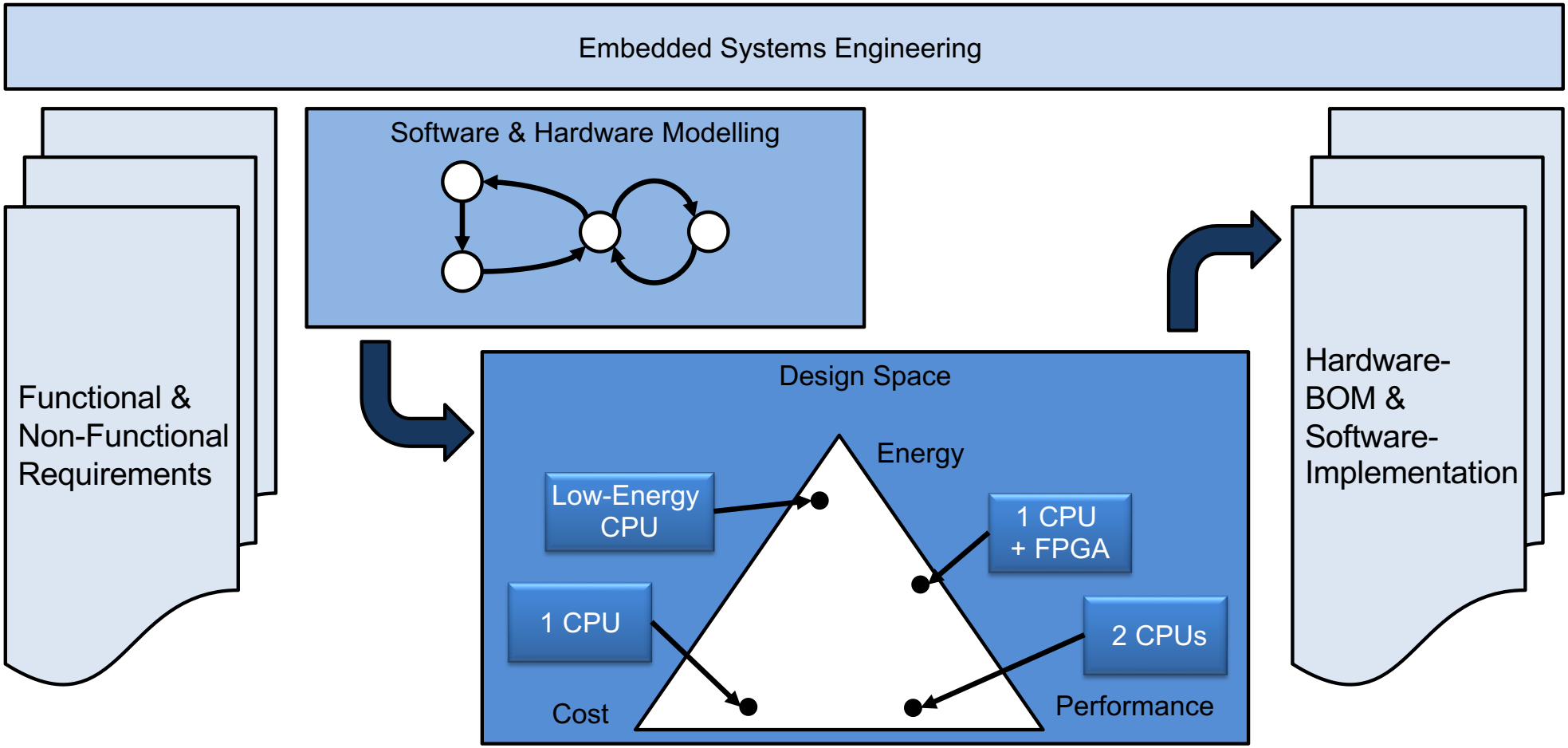
HS 2024

Learning Objectives

After today's lesson, you should be able to:

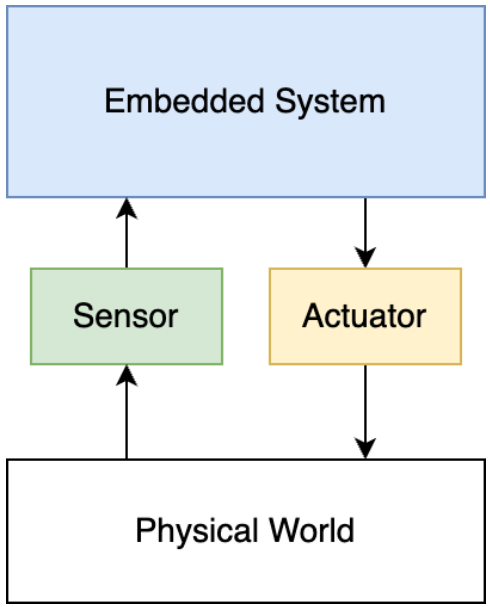
- Explain the benefits of different embedded software paradigms
- Select a suitable software paradigm for your use case
- Recognize a software paradigm when looking at source code
- Design a time-driven embedded system and
- Prove it meets the timing requirements

Advance Organizer ESE



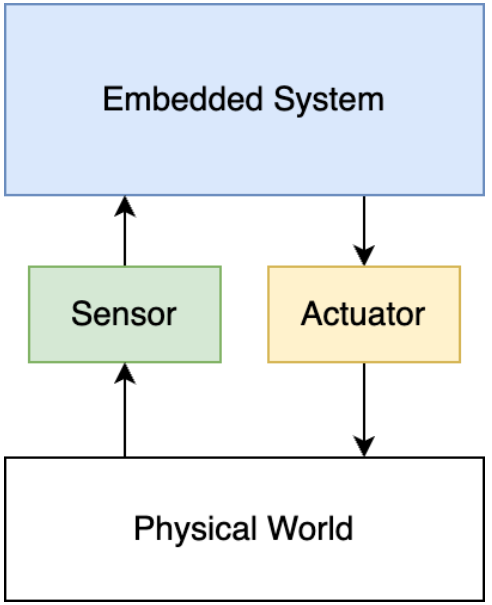
Cyber-physical systems (Embedded Systems)

- Embedded systems can be seen as an interaction between the physical world and computation
- Requirements and constraints are application-specific
- Common components: Sensors, Computer, Actors



Cyber-physical systems

- Some ES…
- … must evaluate and control hundreds of processes simultaneously
- … have extremely tight reaction times
- … control many instances of the same object
- … need to be very interactive
- … have very relaxed timing requirements



- What can guide us in developing the embedded system?
- ➔ embedded software paradigms

Software Paradigms

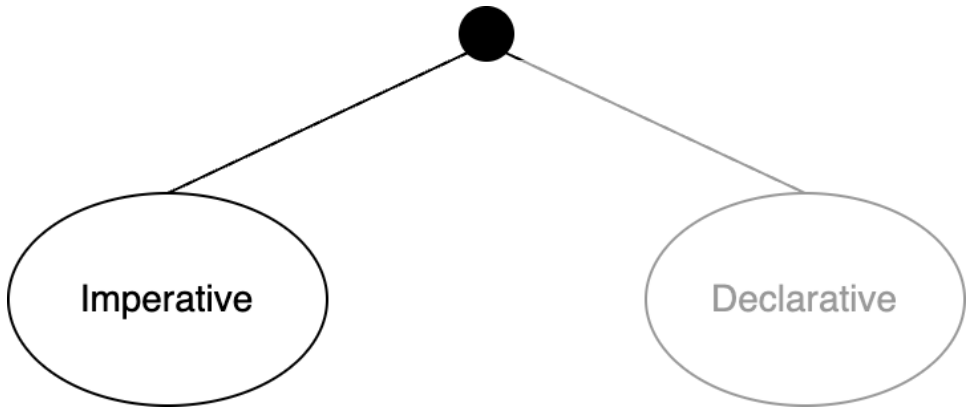
- Developer's Task: translate a design into software
- Software execution changes the state of the embedded system:
  - The processor executes machine code instructions that were compiled from high-level programming languages
- Software describes
  - what must change in an embedded system
  - and, usually, how the change is to be made
- Embedded software may have real-time requirements, adding complexity:
  - when the change needs to happen
  - how often a change may be required
  - how quickly or by what deadline

Software Paradigms

- Software paradigms add guidelines and can help improve software quality
- Two main paradigms:
  - Imperative: define HOW to do something
  - Declarative: define WHAT to do

Software Paradigms: Imperative

- Imperative software consists of a **sequence of commands** that need to be executed to achieve the required change in the system.
- Step-by-step instructions on how to solve the task
- Performance closely defined by the machine architecture:
  - The compiler needs to know the processor (instruction set architecture)
  - The developer needs to know the system architecture

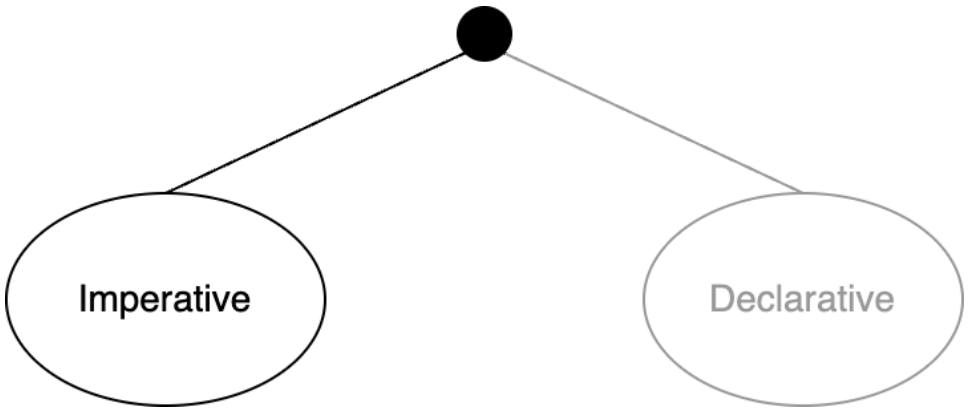


C, C++, Java, Python

Software Paradigms: Imperative

- Imperative programming languages…
- … are easy to understand: small, specific steps
- … are very detailed: many lines of code needed
- … have a defined order: commands are executed in sequence
- … can have side effects (memory contents are changed several times until the task is finished, i.e. in a for-loop)

```
int sum = 0, nums = [1..10];
for (i = 0; i < 10; i++){
  sum = sum + nums[i]; //side
effect!
}
// execution:
sum = sum + 1; i = i + 1;
sum = sum + 2; i = i + 1; …
```

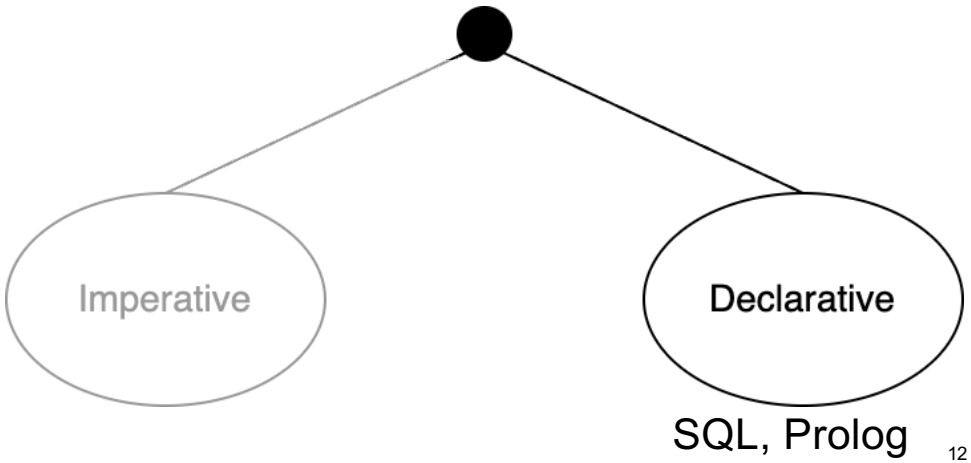


Software Paradigms: Declarative

- Declarative programming describes **what** needs to be done:
  - specifying the rules, instead of specifying the steps.
- By describing **what** needs to be done, the **how** is left to the language or the compiler.
- Tasks are described as rules or logic that the program should follow.
- Declarative programming languages aim to have no side effects.

```
sum[1..10]
// definition of patterns
sum::[Int] --> Int
sum[n] = n
sum(n:ns) = n + sum(ns)

// execution:
sum[1,2,3,4,5,6,7,8,9,10]
  = 1 + sum[2,…,10]
  = 1 + (2 + sum[3,…,10]) …
```



Imperative vs Declarative

- **Imperative programming mainly focuses on:**

a) What needs to be done

b) How tasks are accomplished through steps

c) Only real-time tasks

d) Hardware configuration

Imperative vs Declarative

- **What would be a likely drawback of declarative programming in embedded systems?**

A) Too many lines of code

B) Poor support for timing and control over execution

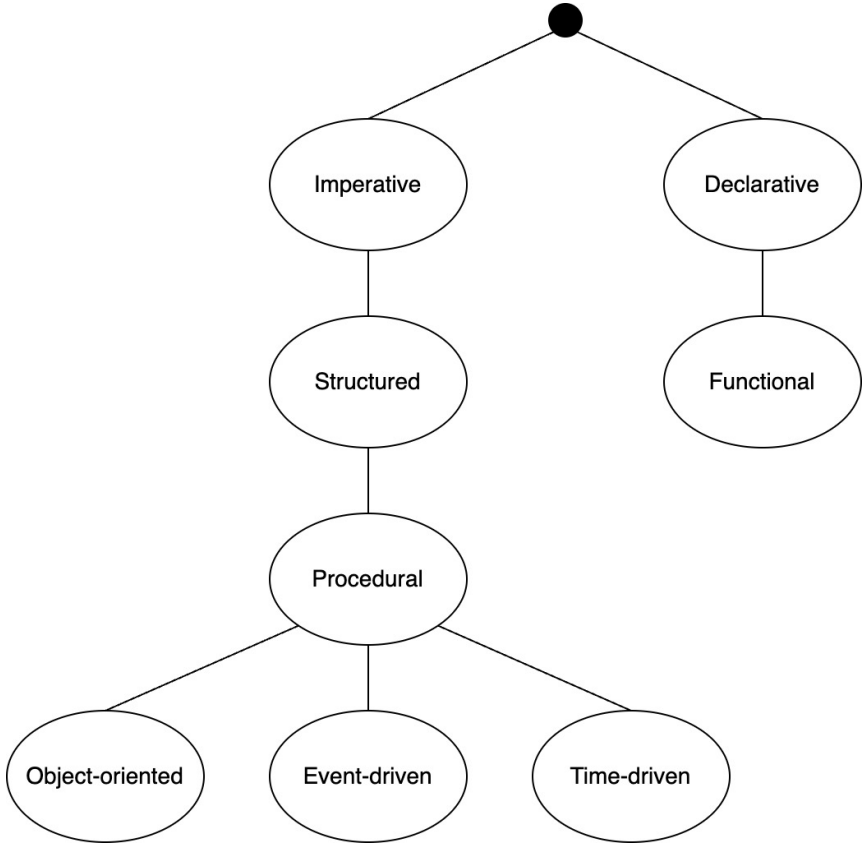
C) More side-effects

D) Always slower performance

Imperative vs Declarative

Feature	Imperative	Declarative
Focus	How to perform a task	What the outcome should be
Control	Step-by-step, explicit instructions	Rules and logic
Side effects	Commonly present	Usually avoided
Timing support	Good, timing and multitasking possible	Rare, poor control over execution timing
Example in Emb. Sys	C for a traffic light control system	SQL for configuration, logic programming

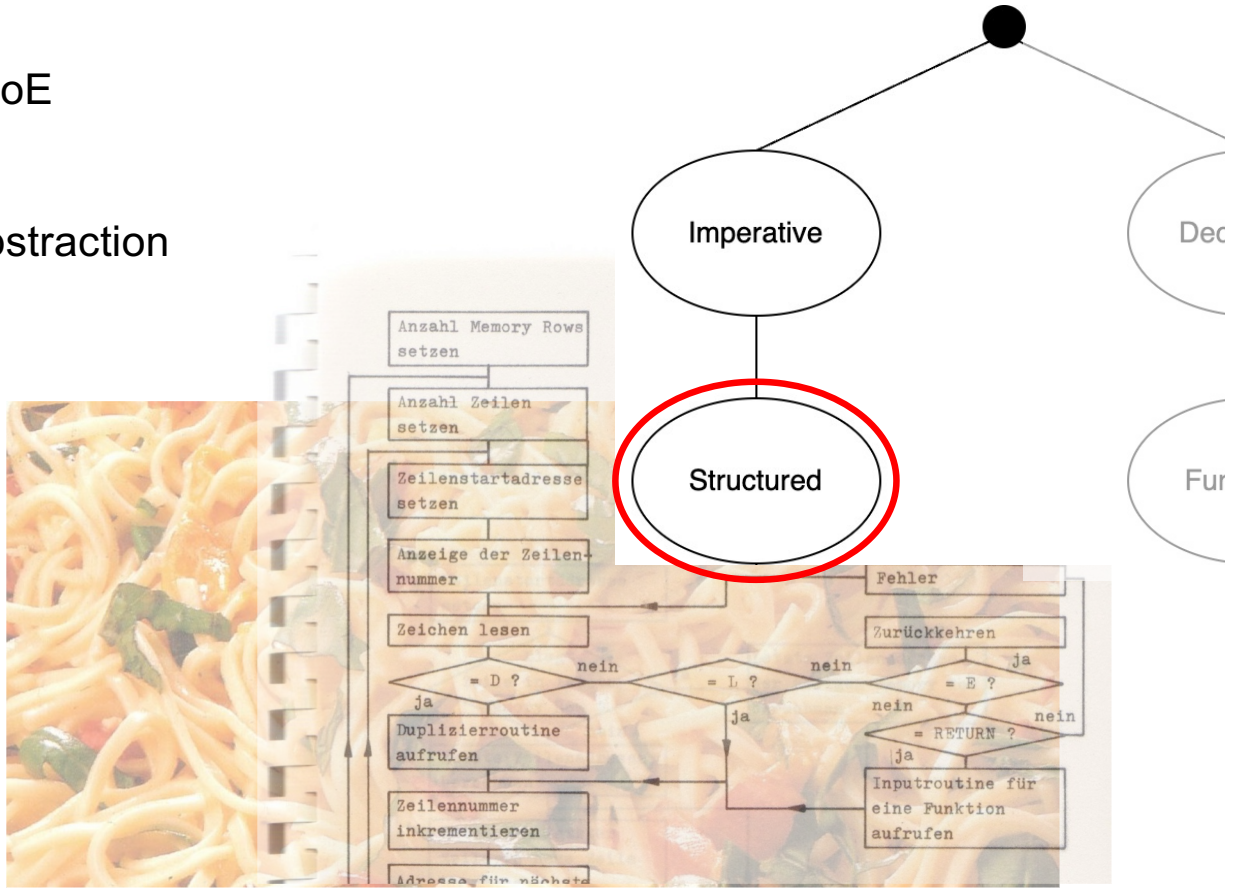
Down the Rabbit Hole…



Structured Paradigm

Zürcher Hochschule für Angewandte Wissenschaften
zh aw School of Engineering
InES Institute of Embedded Systems

- Well-known to any student at SoE
- Giving structure to a program
- Allowing for a higher level of abstraction



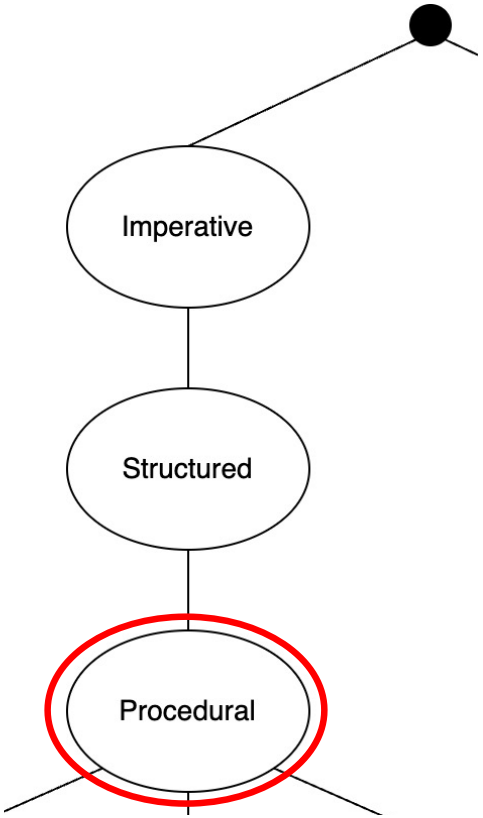
Procedural Paradigm

Zürcher Hochschule für Angewandte Wissenschaften
zh aw School of Engineering
InES Institute of Embedded Systems

Giving more structure to a program:

Using procedures to break a program into small tasks…

- … also called subroutines or functions
- … allows to fully test the software in small parts: unit testing for individual functions



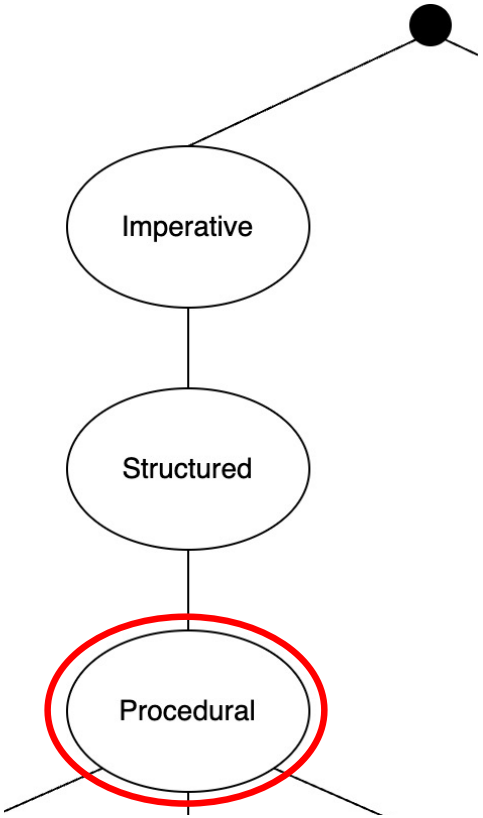

```
int add(int a, int b) {
    return a + b;
}
```

Procedural Paradigm

Zürcher Hochschule für Angewandte Wissenschaften
zh aw School of Engineering
InES Institute of Embedded Systems

Data storage and procedures

- Procedures don’t contain the data they process
  - except for static variables
  - local variables are lost after execution
- Data is separated from procedures
  - variables
  - structs

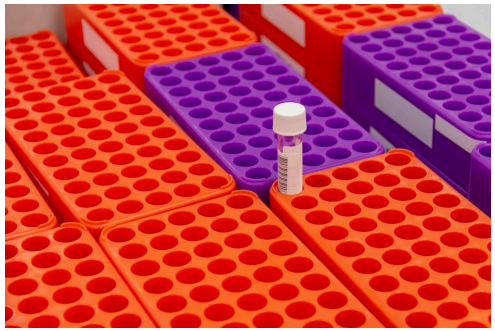


Separation of data and function

```c
// Data:
struct sample_holder {
  int num_rows;
  int num_cols;
  char serial_nr[20];
};

int main() {
// Create an instance of the vial_holder struct
  struct vial_holder holder1;
  // Assign values to the struct members
  holder1.num_rows = 8;
  holder1.num_cols = 12;
  sprintf(holder1.serial_nr, "VH2024-001");
```

Visienco

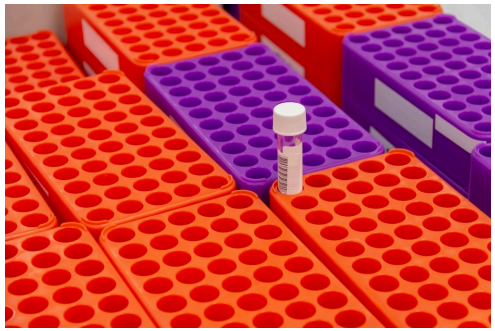


Colourbox

Separation of data and function

```c
// Function:
void print_sample_holder(struct sample_holder holder) {
  printf("Vial Holder Information:\n");
  printf("Number of rows: %d\n", holder.num_rows);
  printf("Number of columns: %d\n",
         holder.num_cols);
  printf("Serial Number: %s\n", holder.serial_nr);
  printf("Total capacity: %d vials\n",
         holder.num_rows * holder.num_cols);
}
```

Visienco



Colourbox

Procedural Paradigm

Why is unit testing often easier in procedural programming?

A) Data and functions are tightly coupled

B) Everything is event-driven

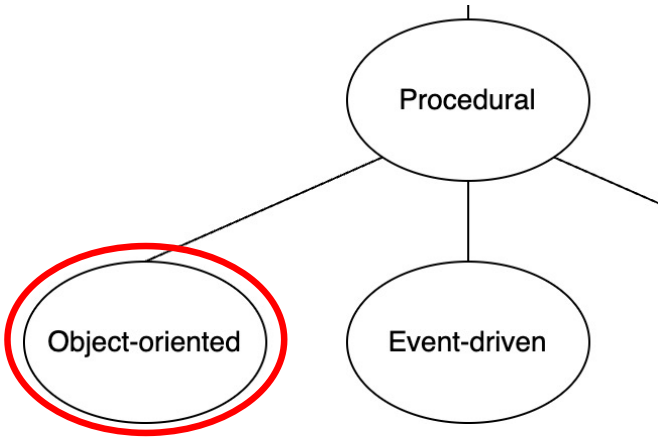
C) Each function operates on the data it is given and can be tested independently

D) Functions are run in parallel

Object-oriented Paradigm

Data has linked procedures

- Data and matching procedures are combined in objects
- Objects are abstract representations,
  - to store defined properties (data)
  - to have procedures operating on the properties
- Similar objects are recognizable as a group through:
  - the same or similar function
  - similar properties
- Similar objects can be distinguished
  - different properties
  - different names



Object-oriented Paradigm

Abstraction: Rocket_Engine

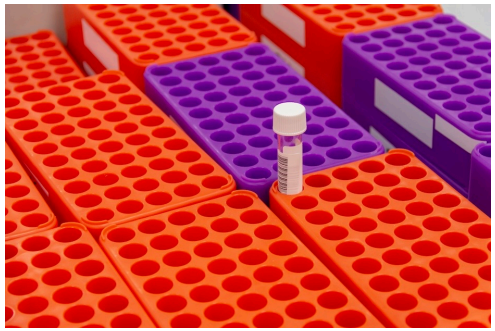
- Each type of Rocket_Engine has its own…
  - max. thrust
  - current thrust
  - thrust vector
  - fuel level
- All Rocket_Engines can…
  - ignite(…)
  - shutdown(…)
  - rud(…)

SpaceX – Starship One

Object-oriented Paradigm

Objects may have

- a role in a system
- attributes (properties, variables)
- a life cycle
- an identity
- states
- connections and relations to other objects

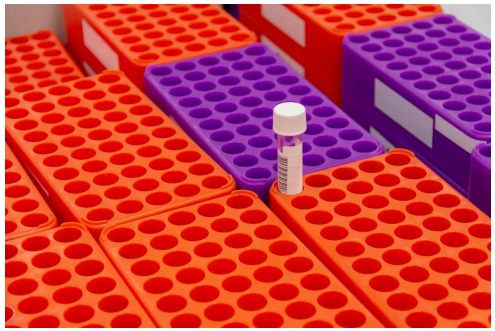


Colourbox

Object-oriented Paradigm

Abstraction of Objects leads to Classes

- Classes represent a set of objects with similar properties ➔ Vial
- Classes combine properties and behaviors
- Classes can be refined (inheritance) ➔ ‘Centrifuge_vial’, ‘Incubation_vial’, …
- Classes inherit properties and behaviors from superclasses
  - all types of vials have a volume, price, color, …
  - some vials may have a max. temperature,

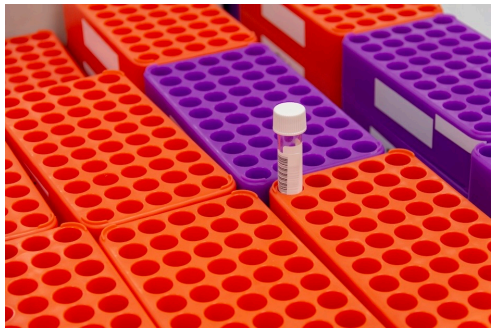


Colourbox

Object-oriented Paradigm

Summary

- Object-oriented paradigm can be useful for ES that comprise many similar objects
- Classes define properties and behaviors
- Classes are organized hierarchically, inheriting properties
- Objects belong to one or more classes
- Objects encapsulate their data



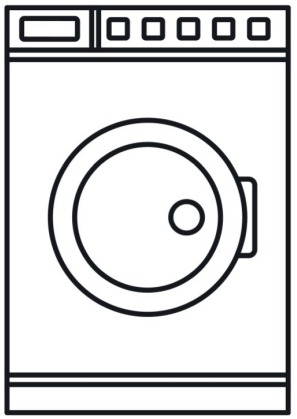
Colourbox

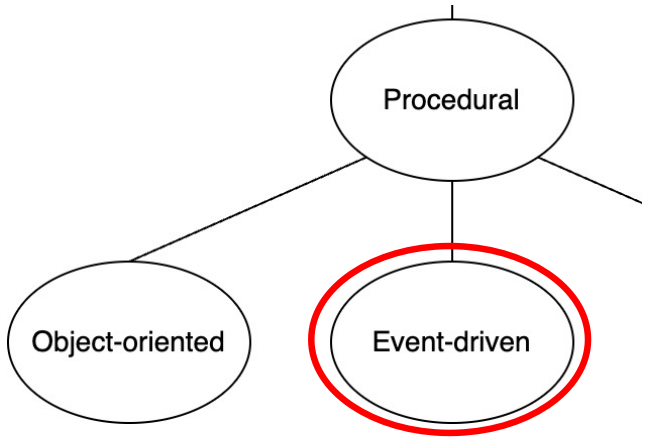
Object-oriented Paradigm

Feature	Procedural	Object-Oriented
Structure	Functions, modules	Objects, classes
Data	Kept separate from functions	Encapsulated with objects
Reuse	Through functions	Through inheritance, polymorphism
Embedded Example	Device drivers, filtering	Systems with multiple device types
Main Advantage	Simplicity, easy testing	Natural modeling, extensibility
Main Limitation	Hard for complex systems	More overhead, complex for small systems

Event-driven Paradigm

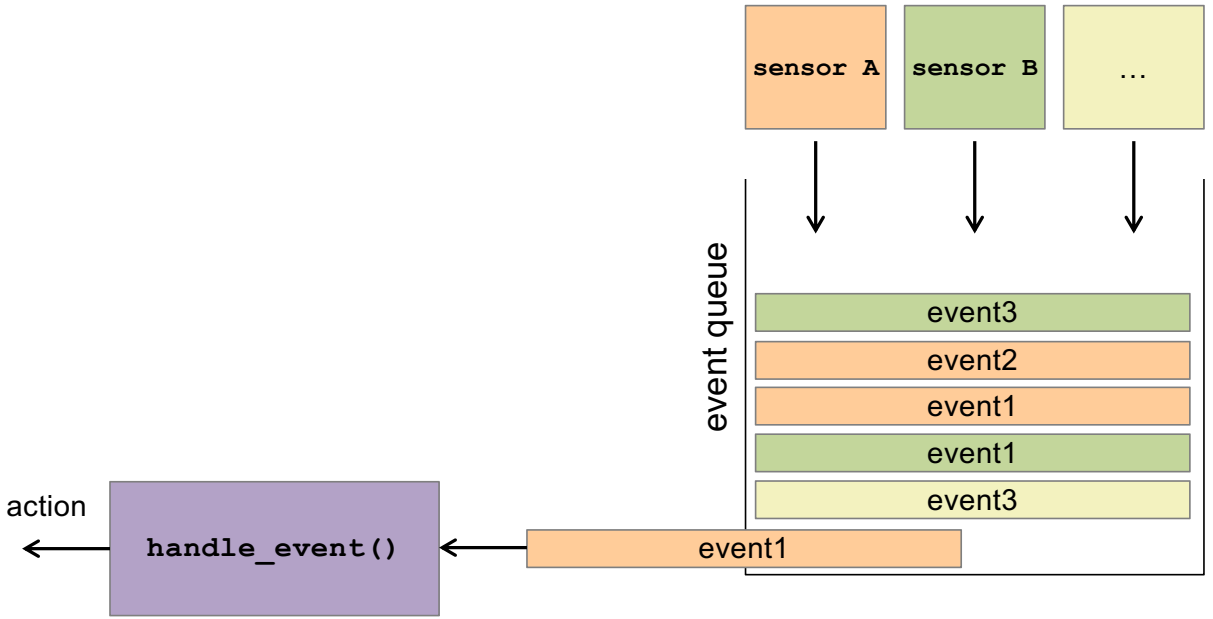
- Embedded systems often need to react to events
- Events may occur periodically, sporadically, or one-shot
- ES may have specifications that set time boundaries for the reaction to events.





Event-driven Paradigm

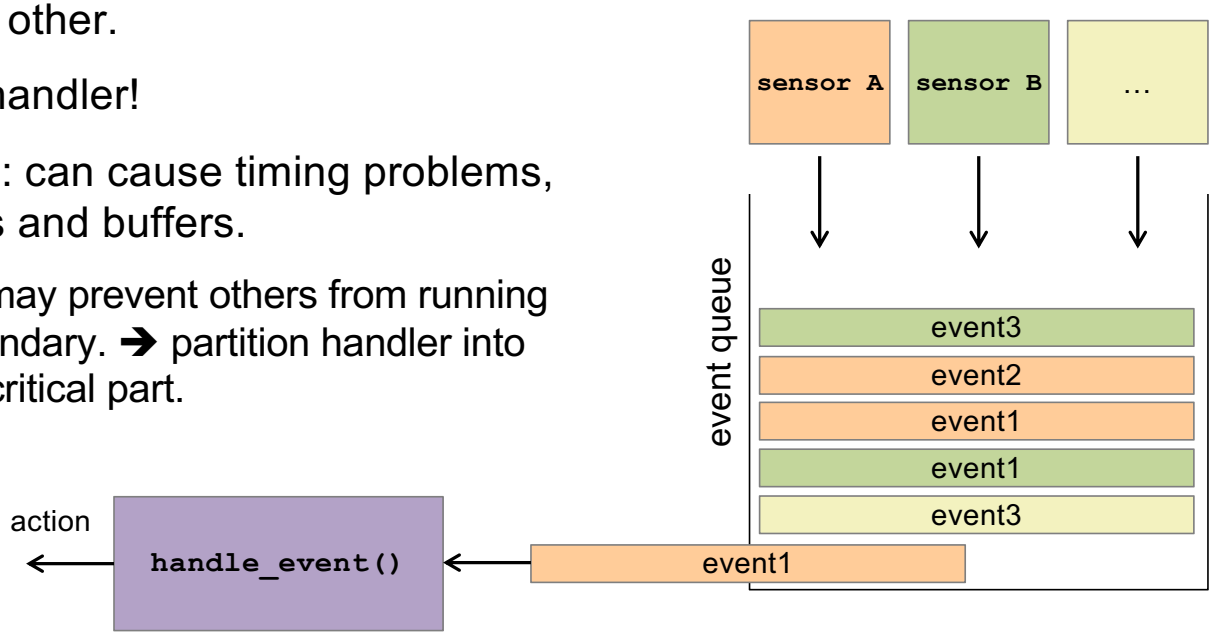
- Every event has a corresponding response.
- Events can be issued by interrupts or internal tasks.
- An event queue can be used to collect all events.



Event-driven Paradigm

Non-preemptive event-triggered scheduling

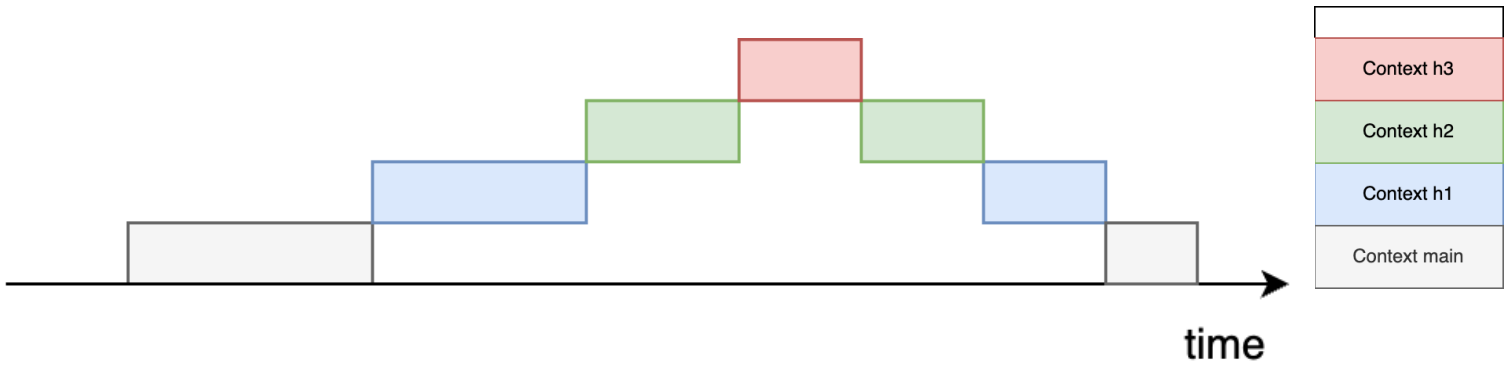
- An event handler checks the event queue and handles one event after the other.
- Handlers do not interrupt each other.
- Interrupts may still preempt a handler!
- dynamic and adaptive reaction: can cause timing problems, conflicts with shared resources and buffers.
  - long runtime of one handler may prevent others from running within their required time boundary. ➔ partition handler into immediate handler and less critical part.



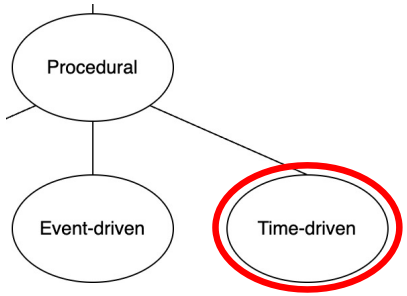
Event-driven Paradigm

Preemptive event-triggered scheduling

- Handlers will interrupt each other.
- The context of the interrupted handler must be stored. This results in a stack frame.
- Handlers must not wait on external events or resources to keep execution times short.
  ➔ no disk access in a handler!
- Protect shared resources (disable interrupts, use semaphores)



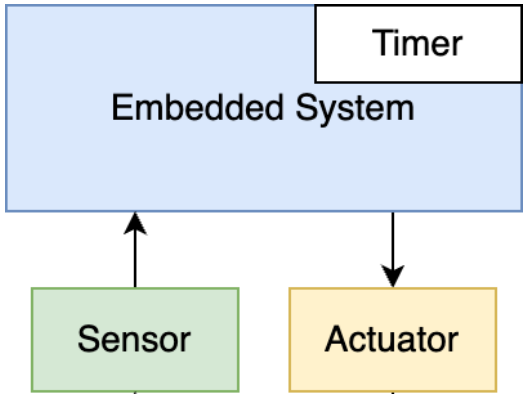
Time-driven Paradigm



Purely time-triggered model

- A timer defines a fixed tick for timing purposes
- No interrupts except tick
- With timing requirements known, a fixed and complex schedule can be used
  → Define a table with the starting times for all handler functions

- Advantage: deterministic timing is feasible within the WCET.
- Disadvantage: Sensors must be polled.

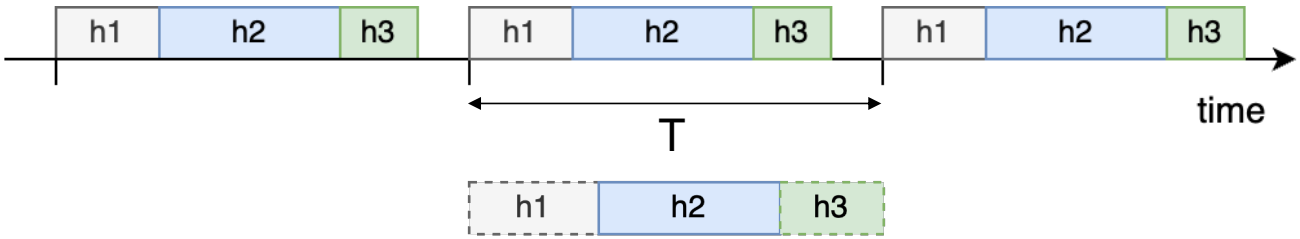


Time-driven Paradigm

Simple periodic time-triggered model

- Regular timer interrupt: period T
- All tasks run with period T

- Execution times are not constant → h2, h3 may have irregular starting times
- No deadlocks to be expected: handlers always run in the same order
- Sum of WCET of all handlers must be smaller than T



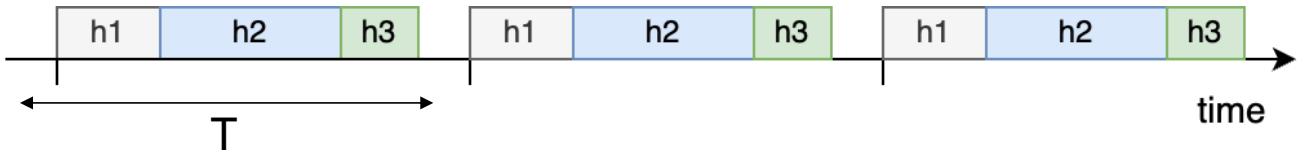
Time-driven Paradigm

Simple periodic time-triggered model

```
counter = 0;
main {
    define task_table;
    set_timer(start_of_first_period);
    while(1) { sleep(); }
}

timer_interrupt_handler {
    counter++;
    set_timer(start_of_next_period);
    for(k = 0; ...){
        run_handler(k);
    }
}
```

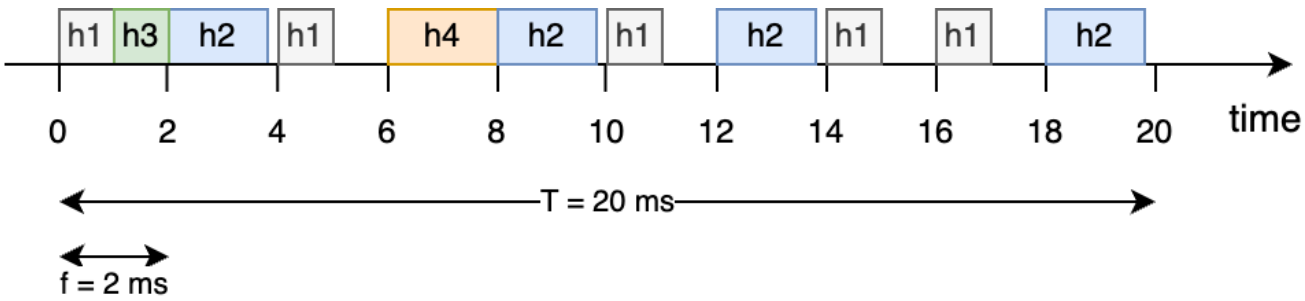
k	h(k)
0	h1
1	h2
2	h3



Time-driven Paradigm

Generic time-triggered scheduler

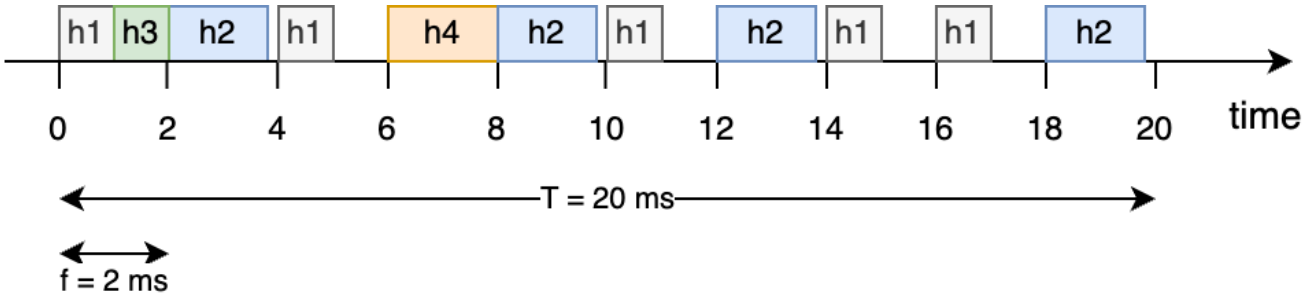
- Handlers may have different periods:
  h1: 4 ms
  h2: 5 ms
  h3: 20 ms
  h4: 20 ms
- Define smaller ticks *f* to provide finer granularity



Time-driven Paradigm

Generic time-triggered scheduler

- define a Task-Descriptor List (TDL) with a cyclic schedule over one period T. (offline)
- TDL contains all activities for the period, with the desired priorities and avoiding deadlocks
- A dispatcher is triggered by a clock tick in every frame. The dispatcher executes the handler(s) as planned in the TDL.



t	h(k)
0	h1, h3
2	h2
4	h1
6	h4
8	h2
10	h1
…	…

Time-driven Paradigm

In a simple periodic scheduler, what is the main requirement for meeting deadlines?

A) All tasks are event-triggered only

B) Interrupts are never allowed

C) Only one handler runs per period

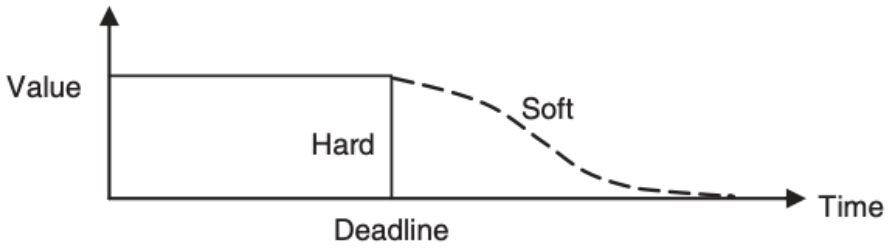
D) The sum of the worst-case execution times fits within the scheduling period

Event-driven vs Time-driven

Feature	Event-Driven	Time-Driven
Trigger Source	External or internal events	Timer interrupts, regular scheduling
Use Case	Keypad, UI, interrupt-driven controls	Motor control, closed-loop periodic tasks
Predictability	Can be unpredictable	Highly deterministic timing
Concurrency	Handlers run as triggered	All handlers scheduled with fixed periods
Drawbacks	Risk of missed deadlines, contention	Sensor polling, inflexible for sporadic tasks

Choose your paradigm

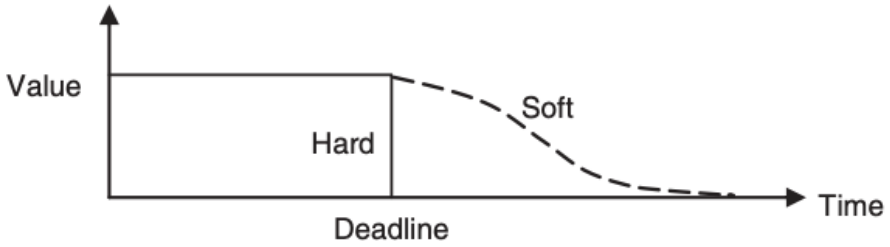
Artem Zakharov , Colourbox



Choose your paradigm

Miroslav.broz - CC BY-SA 3.0, link

- **Hundreds of sensors and actors**
- **Many rectifiers, motors, brakes**
- **Hard and short timing deadlines**



Consequences of paradigms

- **What would change if a traffic light controller used an event-driven instead of a time-driven approach?**

Colourbox – M-Production

Summary

- Software Paradigms define rules and models for software development
- Multiple paradigms may fit the application, they can be mixed
- Multiple paradigms can be combined in different parts of your ES
- There are ways to achieve deterministic timing on an Embedded System
  - fixed schedule, task list